

Algorithm:

1. Start.
2. Read the order n of the square matrix.
3. Check that n is positive. If not, ask again.
4. Declare a variable-length square array $a[n][n]$.
5. Read the matrix elements.
6. Display the menu repeatedly:
 - Check whether main diagonal elements are distinct.
 - Find sum of main diagonal elements.
 - Display diagonal traversal.
 - Exit.
7. Use switch-case to perform the selected operation using separate functions.
8. Continue until the user chooses 4.
9. Stop.

Function 1 – Distinct Main Diagonal Elements

1. Compare every main diagonal element $a[i][i]$ with the diagonal elements after it.
2. If two elements are equal, the diagonal elements are not distinct.
3. Otherwise, all diagonal elements are distinct.

Function 2 – Sum of Main Diagonal

1. Initialize $\text{sum} = 0$.
2. Traverse from $i = 0$ to $n-1$.
3. Add $a[i][i]$ to sum .
4. Display the sum.

Function 3 – Diagonal Traversal

The diagonals are traversed alternately:

- First diagonal: upward-right

- Second diagonal: downward-left
- Third diagonal: upward-right
- Continue alternately.

C PROGRAM:

```
#include <stdio.h>
```

```
/* Function prototypes */
```

```
void checkDistinct(int n, int a[n][n]);
```

```
void diagonalSum(int n, int a[n][n]);
```

```
void diagonalTraversal(int n, int a[n][n]);
```

```
int main()
```

```
{
```

```
    int n, i, j, choice;
```

```
    /* Read and validate the order of the matrix */
```

```
    do
```

```
    {
```

```
        printf("Enter the order of the square matrix: ");
```

```
        scanf("%d", &n);
```

```
        if (n <= 0)
```

```
        {
```

```
            printf("Order must be positive. Enter again.\n");
```

```
        }
```

```
    } while (n <= 0);
```

```

/* Variable-length square array */

int a[n][n];

printf("Enter the matrix elements:\n");

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        scanf("%d", &a[i][j]);
    }
}

/* Menu-driven operations */

do
{
    printf("\n----- MENU ----- \n");

    printf("1. Check whether main diagonal elements are distinct\n");
    printf("2. Find sum of main diagonal elements\n");
    printf("3. Display diagonal traversal\n");
    printf("4. Exit\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:

```

```

        checkDistinct(n, a);
        break;

    case 2:
        diagonalSum(n, a);
        break;

    case 3:
        diagonalTraversal(n, a);
        break;

    case 4:
        printf("Exiting program...\n");
        break;

    default:
        printf("Invalid choice! Please enter 1 to 4.\n");
    }

} while (choice != 4);

return 0;
}

/* Function to check whether main diagonal elements are distinct */
void checkDistinct(int n, int a[n][n])
{

```

```
int i, j;
```

```
int distinct = 1;
```

```
printf("Main diagonal elements: ");
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    printf("%d ", a[i][i]);
```

```
}
```

```
/* Compare the main diagonal elements */
```

```
for (i = 0; i < n - 1; i++)
```

```
{
```

```
    for (j = i + 1; j < n; j++)
```

```
    {
```

```
        if (a[i][i] == a[j][j])
```

```
        {
```

```
            distinct = 0;
```

```
            break;
```

```
        }
```

```
    }
```

```
if (distinct == 0)
```

```
{
```

```
    break;
```

```
}
```

```
}
```

```
if (distinct == 1)
{
    printf("\nAll the main diagonal elements are distinct.\n");
}
else
{
    printf("\nAll the main diagonal elements are not distinct.\n");
}
}
```

```
/* Function to find sum of main diagonal elements */
```

```
void diagonalSum(int n, int a[n][n])
```

```
{
    int i;
    int sum = 0;

    for (i = 0; i < n; i++)
    {
        sum = sum + a[i][i];
    }
}
```

```
printf("Sum of main diagonal elements: %d\n", sum);
}
```

```
/* Function to display elements in diagonal traversal */
```

```
void diagonalTraversal(int n, int a[n][n])
```

```

{
    int d, row, col;

    printf("Diagonal traversal: ");

    /* There are (2*n - 1) diagonals */
    for (d = 0; d <= 2 * n - 2; d++)
    {
        if (d % 2 == 0)
        {
            /*
             * Even diagonal:
             * Traverse upward-right
             */

            if (d < n)
            {
                row = d;
                col = 0;
            }
            else
            {
                row = n - 1;
                col = d - n + 1;
            }

            while (row >= 0 && col < n)
            {

```

```

        printf("%d ", a[row][col]);

        row--;

        col++;

    }

}

else

{

    /*

    * Odd diagonal:

    * Traverse downward-left

    */

    if (d < n)

    {

        row = 0;

        col = d;

    }

    else

    {

        row = d - n + 1;

        col = n - 1;

    }

    while (row < n && col >= 0)

    {

        printf("%d ", a[row][col]);

        row++;

        col--;

```

```

    }
}

printf("\n");
}

```

TEST CASE TABLE:

TEST CASE	INPUT MATRIX	CHOICE	EXPECTED RESULT
1	1 2 3 / 4 5 6 / 7 8 9	1	Diagonal 1 5 9 → all distinct
2	1 2 3 / 4 5 6 / 7 8 9	2	Sum = 15
3	1 2 3 / 4 5 6 / 7 8 9	3	1 2 4 7 5 3 6 8 9
4	1 2 3 / 4 5 6 / 7 8 9	1	Diagonal 1 1 9 → not all distinct
5	5	1	5 → distinct
6	5	2	Sum = 5
7	Any valid matrix	5	Invalid choice message
8	Any valid matrix	4	EXIT